

A question brought to an OZ expert

Understanding and mitigating HTTP 429

The team was seeing throttling.
Would a second subscription help?

Understand 429 > explore mitigations > compare them in a demo.

Take these slides and Python / C# samples back to your team.

Pablo Piovano

Microsoft MVP · Docker Captain · OZ

Start here

HTTP 429: Too Many Requests

The service is limiting requests right now.
The limit depends on the model and deployment.

Service in Microsoft Foundry	Rate limits to check
Azure OpenAI	Requests per minute (RPM) and tokens per minute (TPM).
Claude	Requests, uncached input tokens and output tokens per minute.
Microsoft MAI	MAI-Thinking-1: RPM and TPM. Check other MAI models separately.

A 429 is a signal, not a diagnosis. Read the error and retry guidance before choosing a fix.

Microsoft Learn: [Azure OpenAI](#) · [Claude](#) · [MAI-Thinking-1](#)

Why am I getting a 429?

Request rate and token rate describe different constraints

RPM

How many requests arrive
in a time window.

1 2 3 4 5 6

Short bursts can matter even when an average looks low.

TPM

How much token demand
requests create over time.

Prompt Output allowance

A small number of large requests can create high demand.

HTTP 429 is a signal to investigate the applicable limit.

[Source: Microsoft Learn · Azure OpenAI quotas and limits](#)

Mitigations in our applications

Control each model call

Control	What to implement	Tradeoff
Bounded retries	Honor retry guidance; use backoff with jitter and a deadline.	Extra attempts and waiting
Pacing + queue	Space new sends; bound the queue and its wait time.	Some work waits or is rejected
Concurrency limit	Cap in-flight calls with a semaphore or worker pool.	Fewer simultaneous calls

Coordinate all app instances sharing a limit. Concurrency alone does not control RPM.

[Source: Azure Architecture Center · Rate Limiting pattern](#)

Why can retries make things worse?

Retries add attempts to the original workload

6 logical requests became 11 HTTP attempts

All six completed in this saved mock run.

The extra attempts also added waiting and traffic.

Observed teaching example from the local retry scenario. Counts can vary.

Respect retry guidance. Bound attempts and total time. Avoid stacked retry policies.

[Source: Azure Architecture Center · Retry pattern](#)

Mitigate token demand: check what the service counts

Quota, estimates and returned usage answer different questions

Quota allocation What capacity can this deployment use?

Admission estimate How much work might this request require?

Returned usage What token usage did the response report?

Reduce unnecessary prompt content and right-size output limits. Verify quality and provider accounting.

[Source: Microsoft Learn · Manage Azure OpenAI quota](#)

How do we verify the cause?

A useful diagnosis connects the error to the workload

1. **Capture the response** Status, service error, retry guidance and available limit headers.
2. **Describe the request pattern** Arrival times, concurrency, prompt size and output allowance.
3. **Check the relevant deployment** Provider, model, region, deployment type and configured limits.
4. **Repeat with one controlled change** Keep enough evidence to compare the outcomes.

[Source: Microsoft Learn · Quota troubleshooting](#)

How do we know a change helped?

Completion alone does not describe the whole outcome

Compare the outcome

- Completed generations
- HTTP attempts and 429s
- End-to-end latency

Keep the workload comparable

- Same provider and input
- Same output allowance
- Same service conditions

Our mock demonstrates RPM behavior. Collect your own Azure evidence separately.

Using the strategies together

Prevention first, recovery second

Strategy	Role	Expected effect
Burst	Baseline or intentional traffic shape	Highest exposure to request-rate rejection
Retry with backoff	Recover an eligible 429 within a budget	May improve completion but adds attempts and time
Paced requests	Space every send, including retries	Reduces burst pressure but cannot guarantee zero 429s

Our Paced scenario combines pacing with bounded retries. Use Burst only when the workload and service can accept it.

Azure Architecture Center: [Rate Limiting pattern](#) · [Retry pattern](#)

Inside the sample code

Retry only while the budget allows it

Source excerpts: stop when attempts or the deadline are exhausted.

Python

```
if status != 429 or attempt == retries:
    break
if time.monotonic() + delay >= deadline:
    outcome = "retry_wait_exceeds_deadline"
    break
time.sleep(delay)
```

[Open lab.py on GitHub >](#)

C# / .NET 10

```
if (delay is null) break;
if (watch.Elapsed.TotalSeconds + delay.Value >=
options.Deadline) { outcome =
"retry_wait_exceeds_deadline"; break; }
await Task.Delay(TimeSpan.FromSeconds(delay.Value));
backoff += delay.Value;
```

[Open Runner.cs on GitHub >](#)

The delay helper reads retry headers, then falls back to exponential backoff with jitter.

Delay helpers: [Python retry_delay](#) · [C# Protocol.RetryDelay](#)

Inside the sample code

Pacing reserves the next send time

Source excerpts inside each Gate admission method; locking and wait loop omitted.

Python

```
if delay <= 0:
    self.sent += 1
    self.history.append(now)
    if self.rpm:
        self.next_at = now + 60 / self.rpm
    return now - began
```

[Open lab.py on GitHub >](#)

C# / .NET 10

```
if (delay <= 0)
{
    sent++; history.Enqueue(now);
    if (rpm > 0) next = now + 60 / rpm;
    return now - began;
}
```

[Open Gate.cs on GitHub >](#)

60 / RPM spaces admissions. Every attempt, including retries, passes through this local gate.

10-minute demonstration

Let's compare the mitigations

Strategy	Completed	HTTP attempts	429s
Burst: no control	2 / 6	6	4
Retry with backoff	6 / 6	12	6
Paced requests	6 / 6	6	0

Same workload: 6 requests, 6 workers, 128 token output cap. Retry recovers by adding attempts; pacing avoids the rejection instead. Local RPM simulation; results can vary.

[Get the demo instructions](#)

Follow the repository instructions to run the interactive demo.

Do we need more quota or another subscription?

The next action depends on the bottleneck we measured

Evidence

Next investigation or action

Bursts coincide with 429s

Evaluate pacing and admission control.

Retries amplify traffic

Review retry ownership, budgets and backoff.

Demand reaches a verified limit

Evaluate quota or capacity options for that deployment.

The cause remains unclear

Collect the missing service and workload evidence.

A second subscription is an architecture choice that needs its own capacity evidence.

Discussion

Take this back to your team.

Share the slides. Run the samples.
Apply the approach to your own workload.

Pablo Piovano

Thank you. Questions?

[Get the slides, Docker instructions and Python / C# code on GitHub](#)